



Mecanismo de detección automática de deriva arquitectónica mediante el uso de *Fitness Functions* y análisis estático de grafos de dependencia: un enfoque de gobernanza continua para PyMES de software.

Juan José Medicis Zambrano

Anteproyecto presentado como requisito parcial para optar al título de:
Magister en Ingeniería de Software

Director(a):
MsC Juan Camilo Parra Martinez

Pontificia Universidad Javeriana Cali
Facultad de Ingeniería y Ciencias
Departamento de Electrónica y Ciencias de la Computación
Cali, Colombia
29 de mayo de 2026

Abstract

Architectural drift is one of the main causes of structural software deterioration in SMEs, because design decisions progressively diverge from the actual implementation when no continuous verification mechanism exists. This proposal aims to develop and experimentally evaluate an automated mechanism for detecting architectural drift based on static code analysis, dependency graph construction, and the execution of *Fitness Functions*, in order to support technical governance without relying on external architectural documentation. The work will be limited to a single technological ecosystem and will evaluate three specific conformance dimensions: layer integrity, absence of dependency cycles, and control of excessive outgoing dependency concentration through explicit thresholds. Based on these rules, the mechanism will generate a structural representation of the system, calculate a Conformance Rate (CR), and classify the architectural state as acceptable, under observation, or non-conforming. The experimental demonstration will compare a baseline version of the system with altered versions containing controlled drifts. The expected outcomes include a functional prototype, a documented rule set, and evidence of the usefulness of the approach to support maintenance and refactoring decisions in agile development environments. In this way, the project seeks to contribute a practical and novel governance mechanism for SMEs by combining code-only evidence, dependency graphs, executable architectural rules, and explicit conformance thresholds.

Resumen

La deriva arquitectónica representa una de las principales causas del deterioro estructural del software en PyMES, debido a que las decisiones de diseño se desalinean progresivamente de la implementación real sin que existan mecanismos continuos de verificación. Este anteproyecto propone desarrollar y evaluar experimentalmente un mecanismo automatizado de detección de deriva arquitectónica basado en análisis estático de código, construcción de grafos de dependencia y ejecución de *Fitness Functions*, con el fin de apoyar la gobernanza técnica sin depender de documentación arquitectónica externa. El trabajo se enfocará en un único ecosistema tecnológico y evaluará tres dimensiones concretas de conformidad: integridad de capas, ausencia de ciclos de dependencia y control de concentración excesiva de dependencias salientes mediante umbrales explícitos. A partir de estas reglas, el mecanismo generará una representación estructural del sistema, calculará una Tasa de Conformidad (CR) y clasificará el estado arquitectónico como aceptable, en observación o no conforme. La demostración del proyecto se realizará mediante un experimento controlado que compare una línea base con versiones alteradas del mismo sistema. Se espera como resultado un prototipo funcional, un catálogo documentado de reglas y evidencia sobre la utilidad del enfoque para apoyar decisiones de mantenimiento y refactorización en contextos de desarrollo ágil. Con ello, el proyecto busca aportar una práctica de gobernanza arquitectónica más continua, objetiva y viable para organizaciones con recursos limitados.

Índice

1. Planteamiento y justificación del problema	6
1.1. Planteamiento del problema	6
1.2. Justificación del proyecto	7
2. Objetivos del proyecto	9
2.1. Objetivo General	9
2.2. Objetivos específicos	9
3. Alcance del proyecto	10
4. Marco teórico y antecedentes	12
4.1. Bases teóricas	12
4.2. Antecedentes	13
5. Gestión y Mitigación de Riesgos Éticos	17
6. Metodología del proyecto	19
6.1. Diseño metodológico	19
6.2. Fases de ejecución	20
6.3. Técnicas e instrumentos de recolección y análisis	22
6.4. Mecanismo de evaluación y demostración	22
6.5. Resultados esperados	23
6.6. Criterios de cierre	24
6.7. Criterios de éxito	25
7. Cronograma	26

Índice de figuras

Índice de tablas

1.	Comparación de antecedentes: aportes y brechas frente al presente proyecto	16
2.	Trazabilidad entre objetivos específicos y resultados esperados	24
3.	Cronograma mensual propuesto para el anteproyecto	27

1. Planteamiento y justificación del problema

1.1. Planteamiento del problema

En el ecosistema actual de desarrollo de software, la arquitectura ha dejado de ser un plano estático para convertirse en una entidad dinámica que evoluciona en cada iteración. Ese dinamismo da origen a la deriva arquitectónica, entendida como la divergencia progresiva entre la arquitectura prescrita y la arquitectura realmente implementada en el código fuente (Martín y col., s.f.). En las PyMES de software este fenómeno es especialmente crítico porque el conocimiento arquitectónico suele permanecer distribuido entre código, configuraciones y decisiones tácitas del equipo, en lugar de estar consolidado en documentación viva y verificable (Anthony y col., 2024; Hamed y col., 2023; Manglaras y col., 2026).

La literatura reporta que, para 2021, cerca del 65 % del presupuesto de TI de una organización se destinaba al mantenimiento de software (Pigazzini, 2021). Llevado a una referencia ilustrativa de COP 100 millones anuales, esto equivale a COP 65 millones usados solo para sostener el sistema existente. Además, la erosión arquitectónica puede incrementar el esfuerzo y el costo de desarrollo entre un 20 % y un 40 %; en esa misma referencia, el sobre costo potencial sería de COP 20 a COP 40 millones adicionales (Jolak y col., 2025; Pigazzini, 2021). En otras palabras, el problema no es abstracto: la pérdida de integridad arquitectónica se traduce en dinero, tiempo de mantenimiento y menor capacidad de cambio.

Cuando se alude a conceptos técnicos como dependencias cíclicas, componentes Dios o violaciones de capas, en realidad se está nombrando problemas muy concretos. Una dependencia cíclica aparece cuando dos o más módulos terminan dependiendo entre sí y ya no pueden cambiarse o probarse de forma aislada. Un componente Dios es un módulo que concentra demasiadas responsabilidades y demasiadas dependencias salientes, por lo que cualquier cambio en él afecta muchas partes del sistema. Una violación de capas ocurre cuando un módulo rompe la separación esperada entre presentación, negocio y persistencia, debilitando la modularidad y el control del diseño (Klein y col., 2022; Sas & Avgeriou, 2022). Estos hallazgos son problemáti-

cos porque dificultan las pruebas, aumentan el tiempo de comprensión del sistema, multiplican los efectos colaterales y vuelven más costosa la refactorización.

Sin mecanismos automatizados de vigilancia, estas desviaciones permanecen ocultas hasta que el sistema ya exhibe lentitud para cambiar, fragilidad en el mantenimiento o dependencia excesiva del conocimiento informal del equipo. Por tanto, el problema de investigación no consiste solo en detectar *smells* arquitectónicos aislados, sino en determinar si es posible establecer una gobernanza continua basada exclusivamente en el código fuente, con reglas explícitas y umbrales operativos que permitan distinguir cuándo una arquitectura se mantiene en un estado aceptable y cuándo requiere intervención (Klein y col., 2022; «Make your architecture agile with fitness functions | Okoone», s.f.).

En ese contexto, la pregunta de investigación que orienta este anteproyecto es la siguiente: *¿es posible diseñar y evaluar un mecanismo automatizado, basado en análisis estático, grafos de dependencia y Fitness Functions, que permita detectar tempranamente derivas arquitectónicas y clasificar el estado de conformidad estructural de un sistema de software para PyMES sin depender de documentación arquitectónica externa?*

1.2. Justificación del proyecto

En ausencia de un mecanismo automatizado de detección de deriva arquitectónica, las PyMES de software continúan operando bajo un modelo reactivo de gobernanza técnica, donde la arquitectura solo se revisa cuando ya existen fallos visibles en el sistema. Esta falta de visibilidad estructural genera una acumulación progresiva de desviaciones entre la arquitectura diseñada y la implementada, fenómeno que no siempre se manifiesta primero como error funcional, sino como deterioro silencioso de la mantenibilidad, de la capacidad de evolución y del costo de introducir cambios (Jolak y col., 2025).

La importancia del proyecto radica en que propone intervenir antes de que ese deterioro se convierta en una trampa de legado. Su aporte no es solamente listar hallazgos técnicos, sino convertir decisiones arquitectónicas que hoy viven de forma

tácita en verificaciones automáticas, repetibles y comprensibles para el equipo. Esto es valioso para las PyMES porque reduce la dependencia de documentación externa, disminuye la revisión manual y ofrece una base objetiva para priorizar mantenimiento y refactorización con recursos limitados (Hamed y col., 2023; Klein y col., 2022).

Lo novedoso del trabajo está en la combinación de cuatro elementos dentro de un mismo mecanismo: extracción automática de dependencias desde el código fuente, representación estructural mediante grafos, traducción de decisiones de diseño a *Fitness Functions* y evaluación del estado arquitectónico con un umbral explícito de conformidad. El núcleo del proyecto, por tanto, no es solo detectar desviaciones, sino demostrar que una PyME puede contar con una señal temprana, objetiva y accionable sobre la salud estructural de su software sin depender de reconstrucciones manuales extensas.

Los beneficiarios directos del proyecto serán los equipos de desarrollo, arquitectos de software y líderes técnicos que necesiten evidencia objetiva para identificar desviaciones estructurales y priorizar refactorización. Como beneficiarios indirectos se encuentran las PyMES de software que requieren mecanismos de gobernanza arquitectónica de bajo costo de adopción, con menor dependencia de documentación externa y mayor capacidad para sostener la evolución de sus sistemas en el tiempo. En consecuencia, la utilidad del proyecto no se limita a un artefacto técnico, sino que se proyecta como apoyo a decisiones de mantenimiento, reducción de riesgo estructural y mejora de la sostenibilidad del producto software.

1.2.1. Hipótesis de trabajo

Si se implementa un mecanismo automatizado que extraiga dependencias desde el código fuente, ejecute *Fitness Functions* sobre un grafo estructural y clasifique los resultados mediante umbrales explícitos de conformidad, entonces será posible detectar tempranamente derivas arquitectónicas relevantes en un proyecto de software de PyME, diferenciar de manera objetiva un estado aceptable de uno problemático y producir evidencia suficiente para apoyar decisiones de mantenimiento y refactorización sin depender de documentación arquitectónica externa.

2. Objetivos del proyecto

2.1. Objetivo General

Desarrollar y evaluar experimentalmente un mecanismo automatizado para la detección de derivas arquitectónicas, basado en grafos de dependencia y en la ejecución de *Fitness Functions*, que permita clasificar el estado de conformidad estructural de proyectos de software de PyMES sin depender de documentación externa.

2.2. Objetivos específicos

1. Formular un catálogo acotado de *Fitness Functions* para tres familias de deriva arquitectónica relevantes en PyMES: violaciones de capas, ciclos de dependencia y concentración excesiva de dependencias salientes, dejando explícita la racionalidad de cada regla, su evidencia esperada y su criterio de incumplimiento.
2. Implementar un componente de análisis estático que genere automáticamente grafos de dependencia y topologías de componentes a partir del código fuente y de los archivos de configuración necesarios del sistema seleccionado.
3. Construir un motor de validación que compare el grafo extraído con las reglas definidas, calcule una **Tasa de Conformidad (CR)** y clasifique el resultado mediante umbrales operativos que permitan determinar cuándo la arquitectura está en estado aceptable, en observación o en condición no conforme.
4. Ejecutar un experimento controlado sobre un caso de estudio, comparando una versión base con versiones que contengan derivas introducidas deliberadamente, para medir la capacidad del mecanismo de detectar desviaciones, clasificar correctamente el estado arquitectónico y generar evidencia útil para la toma de decisiones.

3. Alcance del proyecto

El alcance del proyecto comprende todo lo necesario para construir y demostrar un prototipo funcional de detección de deriva arquitectónica en un contexto acotado y viable para una maestría. En consecuencia, el trabajo incluirá las siguientes actividades y entregables:

- **Selección del caso de estudio y del ecosistema tecnológico:** el prototipo se implementará y probará sobre un único ecosistema tecnológico, Java o .NET, y sobre un solo sistema con código fuente disponible, arquitectura en capas y tamaño manejable dentro del tiempo definido para el proyecto. Para la ejecución se priorizará un sistema propio o de código abierto; si el caso de estudio pertenece a una entidad externa, su uso quedará condicionado a la autorización expresa correspondiente y a la restricción del análisis a información estructural necesaria para la investigación.
- **Definición del catálogo de reglas:** se formalizarán reglas ejecutables para tres dimensiones de conformidad arquitectónica: integridad de capas, ausencia de ciclos de dependencia y control de concentración excesiva de dependencias salientes en componentes o paquetes.
- **Construcción del prototipo:** se desarrollará un mecanismo capaz de inspeccionar el código fuente, extraer dependencias relevantes, construir un grafo estructural y ejecutar *Fitness Functions* sobre dicho grafo.
- **Clasificación de resultados:** el prototipo calculará una **Tasa de Conformidad (CR)** y utilizará umbrales operativos para interpretar si la arquitectura evaluada se encuentra en estado aceptable, en observación o no conforme.
- **Ejecución del experimento:** se realizará una evaluación controlada sobre una versión base del sistema y sobre versiones alteradas con derivas inducidas, con el fin de observar variación en la CR, capacidad de detección y utilidad del reporte generado.

- **Entregables:** el proyecto producirá un repositorio con el prototipo funcional, un catálogo documentado de reglas, la matriz de validación, la evidencia del experimento y el informe final de investigación.

Fuera del alcance del proyecto

- * La reconstrucción completa de toda la arquitectura empresarial o de todos los atributos de calidad del sistema; el prototipo se limitará a las tres dimensiones de conformidad definidas.
- * El soporte simultáneo para múltiples ecosistemas tecnológicos o para repositorios heterogéneos dentro de una misma ejecución.
- * El análisis dinámico en tiempo de ejecución, la minería de trazas o la instrumentación del sistema en producción.
- * La corrección automática de las derivas detectadas; el proyecto se concentrará en detección, clasificación y evidencia para la toma de decisiones.
- * El despliegue del prototipo como producto listo para producción o su integración completa en una tubería CI/CD organizacional.

4. Marco teórico y antecedentes

4.1. Bases teóricas

Esta sección fundamenta los conceptos sobre los cuales se construye el mecanismo de detección automatizada y, sobre todo, explica por qué cada uno participa de manera directa en el proyecto.

- **Deriva y erosión arquitectónica:** La deriva arquitectónica es la divergencia progresiva entre la arquitectura prescrita y la arquitectura realmente implementada en el código fuente (Pigazzini, 2021). Este concepto es el punto de partida del proyecto porque define exactamente el fenómeno que se desea observar: la pérdida de integridad estructural cuando el sistema evoluciona sin verificación continua. Para esta investigación no basta con describir la deriva; se requiere volverla observable y medible en un entorno de PyME, donde la degradación arquitectónica suele detectarse tarde (Anthony y col., 2024).
- **Deuda técnica arquitectónica (ATD):** La ATD explica por qué decisiones de diseño aceptables en el corto plazo pueden generar un costo acumulado en mantenimiento, evolución y refactorización (Hamed y col., 2023; Pigazzini, 2021). Su participación en el proyecto consiste en ofrecer el marco que conecta los hallazgos estructurales con una consecuencia de negocio: cuando la arquitectura pierde control, la organización paga más por cambiar. Esta relación justifica que el prototipo no se limite a reportar anomalías, sino que busque proveer evidencia útil para priorizar intervención técnica.
- **Smells arquitectónicos:** Los smells arquitectónicos son manifestaciones concretas de deterioro estructural que involucran componentes, paquetes o capas completas (Pigazzini, 2021; Sas & Avgeriou, 2022). En este trabajo participan como variables observables del problema. Se seleccionan tres familias porque son detectables desde el código fuente y porque afectan directamente la mantenibilidad: violaciones de capas, ciclos de dependencia y concentración excesiva de dependencias salientes en un componente. Esta selección acota el proyecto sin perder

relevancia práctica.

- **Funciones de aptitud (Fitness Functions):** Las *Fitness Functions* son mecanismos automatizados para evaluar de forma objetiva si una característica arquitectónica se preserva en el tiempo («Fitness Functions – Safeguard Architecture with Automated Checks», s.f.). En el proyecto cumplen un papel central porque traducen decisiones de diseño, que normalmente permanecen implícitas, en reglas ejecutables y repetibles. Esta es una de las piezas nucleares del enfoque propuesto: transformar arquitectura deseada en validaciones concretas sobre el código fuente.
- **Análisis estático y grafos de dependencia:** El análisis estático permite extraer hechos del código sin ejecutarlo y los grafos de dependencia permiten representar componentes como nodos y relaciones como aristas (Klein y col., 2022; Pigazzini, 2021). Ambos conceptos participan directamente en la solución porque hacen posible detectar deriva incluso cuando no existe documentación arquitectónica actualizada. En otras palabras, el grafo es la base estructural sobre la cual el prototipo observará, comparará y reportará conformidad.
- **Algoritmos de recorrido y umbrales de interpretación:** Sobre el grafo de dependencias se aplican algoritmos como la búsqueda en profundidad para detectar ciclos y conteos de relaciones salientes para identificar componentes con concentración excesiva de dependencias (Sas y col., 2019). Sin embargo, detectar no es suficiente: el proyecto también necesita interpretar cuándo un resultado es aceptable y cuándo no. Por ello se incorpora el uso de umbrales operativos, tanto a nivel de regla como de la **Tasa de Conformidad (CR)**, de modo que la salida del prototipo no sea solo descriptiva sino útil para la toma de decisiones (Klein y col., 2022; Murphy y col., 2001).

4.2. Antecedentes

La investigación actual presenta diversas estrategias para gestionar la integridad técnica del software. No obstante, cada antecedente cubre solo una parte del problema, y justamente allí se ubica la diferenciación del presente proyecto.

- **ArchUnit:** ArchUnit demuestra que es viable expresar reglas arquitectónicas como verificaciones ejecutables sobre el código Java («Unit test your Java architecture - ArchUnit», s.f.). Su relevancia para este proyecto está en validar la idea de ".arquitectura autoevaluada". La diferencia es que el presente trabajo no se limita a una biblioteca de pruebas ni a una sola semántica de uso, sino que plantea un prototipo de análisis estructural con grafo, clasificación por umbrales y experimento controlado como evidencia de utilidad para PyMES.
- **ArchSync:** ArchSync aborda la sincronización entre arquitectura esperada y comportamiento observado mediante trazas y heurísticas (Diaz-Pace y col., 2011). Su antecedente es importante porque confirma que la sincronización arquitectura-código es un problema real. La diferencia de este proyecto es que privilegia análisis estático sobre el artefacto vivo del software, lo cual reduce dependencia de instrumentación y lo vuelve más viable para equipos pequeños.
- **Gestión de deuda técnica en PyMES:** Hamed y col. (2023) destacan que las PyMES requieren mecanismos de gobernanza livianos, con bajo costo de adopción y apoyados en herramientas automáticas. Este antecedente conecta directamente con el contexto del proyecto y explica por qué el caso de uso no es una gran empresa con gobierno formal, sino equipos con restricciones de tiempo, documentación y presupuesto.
- **Índices de deuda basados en aprendizaje automático:** El ATDI propuesto por Sas y Avgeriou (2022) clasifica la severidad de smells arquitectónicos usando aprendizaje automático. Este trabajo es relevante porque muestra que la deuda arquitectónica puede resumirse en un indicador interpretable. La diferencia es que el presente proyecto no busca un modelo predictivo complejo, sino una métrica de conformidad comprensible y trazable a reglas explícitas, más adecuada para adopción inicial en contextos PyME.
- **Conformidad automatizada en CI/CD:** Klein y col. (2022) señalan que la verificación automatizada de conformidad arquitectónica tiene mayor valor cuando se acerca al flujo cotidiano de desarrollo. Este antecedente participa en el proyec-

to como justificación práctica: si el mecanismo detecta temprano, la remediación es menos costosa. La diferencia es que, en esta investigación, la prioridad no es integrar el prototipo por completo a un pipeline organizacional, sino demostrar experimentalmente que el enfoque produce hallazgos útiles.

- **Modelos de reflexión:** Murphy y col. (2001) introducen una base formal para comparar el modelo extraído del código con el modelo arquitectónico esperado. Este antecedente fundamenta el motor de validación y la noción de conformidad del proyecto. La diferencia es que aquí esa comparación se operacionaliza para un conjunto pequeño de reglas ejecutables y con una salida orientada a clasificar el estado arquitectónico mediante una CR comprensible.

El siguiente cuadro comparativo resume los aportes de cada antecedente y las brechas que subsisten respecto al presente proyecto, con el propósito de identificar con precisión el espacio de contribución que justifica su ejecución.

Tabla 1: Comparación de antecedentes: aportes y brechas frente al presente proyecto

Antecedente	Aportes al campo	Limitaciones / Brechas
ArchUnit («Unit test your Java architecture - ArchUnit», s.f.)	Valida que las reglas arquitectónicas pueden expresarse como verificaciones ejecutables sobre código Java.	Limitado a una biblioteca y un lenguaje; no produce clasificación por umbrales ni experimento controlado; no está orientado a PyMES.
ArchSync (Díaz-Pace y col., 2011)	Confirma la sincronización arquitectura-código como problema real; usa trazas de comportamiento.	Requiere instrumentación del sistema en ejecución; mayor complejidad de adopción para equipos pequeños; no usa análisis estático puro.
Gestión de ATD en PyMES (Hamed y col., 2023)	Evidencia que las PyMES necesitan mecanismos ligeros y automatizados; fundamenta el contexto del trabajo.	No provee artefacto técnico; no define reglas ejecutables ni métricas de conformidad estructural.
ATDI con ML (Sas & Avgeriou, 2022)	Muestra que la deuda arquitectónica puede resumirse en un indicador interpretable usando aprendizaje automático.	Modelo de caja negra con baja trazabilidad; difícil adopción inicial en PyMES sin experiencia en ML.
Conformidad en CI/CD (Klein y col., 2022)	Muestra el valor de la verificación continua integrada al flujo de desarrollo.	Centrado en pipelines maduros; no propone experimento controlado; asume contextos con mayor gobernanza formal.
Modelos de reflexión (Murphy y col., 2001)	Base formal para comparar modelo extraído con modelo esperado; fundamenta el motor de validación.	Sin operacionalización para conjuntos acotados de reglas ejecutables ni clasificación por umbrales comprensibles.
Brecha consolidada	Ningún antecedente combina: (1) análisis estático exclusivo del código fuente, (2) grafo de dependencia + <i>Fitness Functions</i> , (3) clasificación por Tasa de Conformidad (CR), (4) experimento controlado con criterios explícitos, y (5) orientación específica a PyMES sin documentación arquitectónica externa. Esta brecha justifica el presente trabajo.	

En síntesis, los antecedentes confirman que el problema existe, que la automati-

zación es viable y que las reglas ejecutables aportan valor. Lo que diferencia a este proyecto es su foco en PyMES, su dependencia exclusiva del código fuente como evidencia viva, su uso combinado de grafo estructural y *Fitness Functions*, y la incorporación de un experimento controlado con criterios explícitos para determinar si el mecanismo realmente ayuda a distinguir entre una arquitectura aceptable y una arquitectura problemática.

5. Gestión y Mitigación de Riesgos Éticos

El valor del proyecto no depende únicamente de identificar riesgos éticos, sino de establecer desde el diseño del prototipo un conjunto de controles que reduzcan la probabilidad de daño y limiten el uso indebido de sus resultados. En ese sentido, la estrategia ética del anteproyecto se fundamenta en cuatro principios: privacidad por diseño, transparencia del diagnóstico, supervisión humana y uso proporcional de las métricas. Desde la gestión del riesgo, cada situación puede tratarse mediante una de cuatro respuestas: aceptarlo, evitarlo, transferirlo o mitigarlo. A continuación se asigna el tratamiento más adecuado para cada riesgo identificado.

- **Riesgo sobre información sensible. Tratamiento: evitarlo.** El análisis estático requiere acceso al código fuente, por lo que la respuesta más adecuada consiste en impedir desde el inicio escenarios innecesarios de exposición. El prototipo se operará en entornos restringidos, con repositorios autorizados y bajo acuerdos explícitos de confidencialidad. Los artefactos generados se limitarán a relaciones estructurales y métricas agregadas, evitando exponer fragmentos de código, credenciales o lógica de negocio. Además, se priorizará el procesamiento local y la minimización de datos almacenados (Hamed y col., 2023; Modawski & Wolniak, 2025).
- **Riesgo de sesgo algorítmico y opacidad del diagnóstico. Tratamiento: mitigarlo.** Para evitar decisiones de refactorización basadas en umbrales arbitrarios o interpretaciones erróneas, las reglas de evaluación deberán ser explícitas, versionadas y trazables a criterios arquitectónicos previamente definidos. Cada

resultado deberá explicar qué regla se activó, cuál fue la evidencia estructural encontrada y qué severidad se asignó. Además, la validación del prototipo se realizará con revisión humana sobre casos de prueba controlados, de forma que la salida automatizada se utilice como apoyo a la decisión y no como veredicto incuestionable (Klein y col., 2022; Sas & Avgeriou, 2022).

- **Riesgo de impacto sociotécnico en los equipos. Tratamiento: mitigarlo.** Para reducir la posibilidad de que la gobernanza automatizada sea percibida como vigilancia punitiva, el uso de los reportes deberá orientarse al nivel de componente o sistema y no a la evaluación individual de desarrolladores. El mecanismo se incorporará como soporte para conversaciones técnicas, retrospectivas arquitectónicas y priorización de refactorización, no como instrumento disciplinario (Hamed y col., 2023; Paul & Wang, 2019).
- **Riesgo de formalismo vacío. Tratamiento: aceptarlo de forma controlada.** Siempre existirá la posibilidad de que una métrica de conformidad sea usada de manera mecánica y sin reflexión contextual. Ese riesgo no puede eliminarse por completo, por lo que la respuesta más realista es aceptarlo de forma controlada y reducir su impacto mediante revisión contextual, registro de excepciones justificadas y ajustes periódicos de las reglas conforme evolucione el sistema. La **CR** no debe reemplazar el juicio técnico, sino activar discusiones de diseño informadas y verificables (Modawski & Wolniak, 2025).
- **Riesgo de sobreinterpretación de la Tasa de Conformidad (CR). Tratamiento: transferirlo parcialmente a la decisión humana responsable.** La **CR** producida por el mecanismo solo representa el cumplimiento de las reglas configuradas dentro del alcance del prototipo: integridad de capas, ausencia de ciclos de dependencia y control de concentración excesiva de dependencias salientes. Para evitar que la métrica se entienda como veredicto absoluto, cada reporte deberá declarar qué reglas fueron evaluadas, qué módulos fueron analizados, qué umbrales se aplicaron y qué aspectos quedaron fuera del diagnóstico. La decisión final sobre priorización de refactorización o aceptación del riesgo recaerá en el director

del proyecto o en el responsable técnico del caso de estudio, no en la herramienta (Klein y col., 2022; Modawski & Wolniak, 2025).

En conjunto, estas medidas buscan que la automatización propuesta mantenga valor técnico sin desbordar su propósito investigativo ni generar incentivos de uso contrarios a la ética profesional. La mitigación, por tanto, no se plantea como un anexo del proyecto, sino como una condición de validez para su aplicación en contextos reales de PyMES de software.

6. Metodología del proyecto

Este proyecto se desarrollará como una investigación aplicada, con enfoque predominantemente cuantitativo y con estrategia de **Design Science Research (DSR)**, dado que el resultado principal es la construcción y evaluación de un artefacto software orientado a resolver un problema práctico de gobernanza arquitectónica (Hamed y col., 2023; Klein y col., 2022). Para que el trabajo sea alcanzable dentro de los cinco meses y las 192 horas definidas para el anteproyecto, la estrategia DSR se ejecutará mediante ciclos iterativos cortos, cada uno con un entregable verificable y un cierre de revisión con el director.

6.1. Diseño metodológico

El diseño metodológico se apoyará en las siguientes decisiones operativas:

- **Unidad de trabajo:** un único sistema de software con código fuente disponible, arquitectura en capas y tamaño acotado, implementado en un solo ecosistema tecnológico seleccionado al inicio del proyecto (Java o .NET).
- **Artefacto a construir:** un prototipo que extraiga dependencias desde el código fuente, construya un grafo estructural, ejecute reglas de validación arquitectónica y produzca un reporte con hallazgos, evidencia trazable y una **Tasa de Conformidad (CR)**.

- **Alcance técnico del prototipo:** el mecanismo se limitará a tres dimensiones de conformidad: integridad de capas, ausencia de ciclos de dependencia y control de concentración excesiva de dependencias salientes en componentes o paquetes. Esta delimitación mantiene el esfuerzo dentro del tiempo disponible y es coherente con los objetivos específicos del anteproyecto (Anthony y col., 2024; Sas y col., 2019).
- **Umbral operativo de interpretación:** la **CR** se interpretará con tres rangos iniciales para la evaluación experimental: estado aceptable si $CR \geq 85\%$, estado en observación si $70\% \leq CR < 85\%$ y estado no conforme si $CR < 70\%$. Estos umbrales no pretenden convertirse en estándar universal, sino servir como criterio explícito para decidir qué se considera bien o mal dentro del experimento controlado.
- **Estrategia de validación:** el artefacto se evaluará mediante un estudio de caso controlado, ejecutándolo primero sobre una línea base y después sobre versiones con derivas introducidas deliberadamente para verificar su capacidad de detección, la variación de la CR y la utilidad de la clasificación obtenida (Pigazzini, 2021; Sas & Avgeriou, 2022).

6.2. Fases de ejecución

Las fases de ejecución guardan correspondencia directa con los objetivos específicos del proyecto: la primera fase responde al objetivo de definir el catálogo de reglas, la segunda y la tercera al objetivo de construir la base técnica del análisis estático y del grafo estructural, la cuarta al objetivo de implementar el motor de validación con umbrales de interpretación, y la quinta al objetivo de ejecutar el experimento controlado y analizar sus resultados.

1. **Delimitación del caso de estudio y catálogo de reglas:** se seleccionará el ecosistema tecnológico a trabajar, se escogerá el sistema base y se definirá un catálogo acotado de reglas ejecutables. Como resultado se construirá una matriz de validación con la descripción de cada regla, su propósito, la evidencia esperada, el criterio de incumplimiento y, cuando aplique, el umbral particular asociado.

En esta fase se tomarán como referencia las *Fitness Functions* como mecanismo para traducir decisiones arquitectónicas en verificaciones automáticas («Fitness Functions – Safeguard Architecture with Automated Checks», s.f.; Paul & Wang, 2019).

2. **Diseño técnico del prototipo:** se definirá la arquitectura interna del mecanismo, las entradas y salidas del proceso, la estructura del grafo de dependencia y el formato del reporte de resultados. El objetivo de esta fase es reducir riesgo de implementación antes de programar, dejando explícitas las relaciones que se van a analizar, la manera de calcular la CR y la interpretación de los resultados frente a los umbrales definidos.
3. **Construcción del componente de extracción:** se implementará el módulo encargado de inspeccionar el código fuente y los archivos de configuración necesarios para inferir dependencias estructurales. El producto de esta fase será una representación navegable del sistema en forma de grafo, suficiente para consultar paquetes, componentes y relaciones relevantes para el análisis arquitectónico (Klein y col., 2022; Murphy y col., 2001).
4. **Construcción del motor de validación:** sobre el grafo obtenido se implementarán las reglas definidas en la primera fase. Este motor calculará la **CR** como proporción de reglas cumplidas frente al total de reglas evaluadas, clasificará el estado del sistema según los umbrales operativos y reportará las violaciones detectadas por tipo, ubicación y severidad. Con ello se dará cumplimiento al objetivo de automatizar la verificación de conformidad arquitectónica (Paul & Wang, 2019; «Unit test your Java architecture - ArchUnit», s.f.).
5. **Evaluación y análisis de resultados:** el prototipo se ejecutará sobre el caso de estudio seleccionado y sobre versiones con derivas controladas, tales como dependencias entre capas no permitidas, ciclos entre módulos y componentes con concentración excesiva de dependencias salientes. Los resultados se analizarán comparando las derivas esperadas con las efectivamente detectadas, la variación de la **CR**, la clasificación del estado arquitectónico y la utilidad del reporte para

apoyar decisiones de mantenimiento y refactorización (Anthony y col., 2024; Sas y col., 2019).

6.3. Técnicas e instrumentos de recolección y análisis

Para desarrollar y evaluar el prototipo se utilizarán las siguientes técnicas e instrumentos:

- **Revisión bibliográfica focalizada:** para consolidar criterios de deriva arquitectónica y seleccionar reglas viables para un prototipo de alcance acotado.
- **Matriz de reglas arquitectónicas:** en la que se registrará cada regla, su racionalidad, patrón esperado y evidencia de incumplimiento.
- **Bitácora de desarrollo y ejecución:** donde se documentarán decisiones, ajustes del prototipo, resultados de las corridas y observaciones del director.
- **Tablas comparativas de resultados:** orientadas a registrar número de violaciones detectadas, porcentaje de conformidad, tipo de deriva identificada, clasificación del estado arquitectónico y diferencias entre la línea base y las versiones alteradas.

6.4. Mecanismo de evaluación y demostración

La evaluación del proyecto se concretará mediante un experimento controlado de caso único, suficiente para demostrar la utilidad del artefacto dentro del alcance del anteproyecto. La demostración seguirá esta lógica:

1. Seleccionar una versión base del sistema que represente el comportamiento arquitectónico esperado para el caso de estudio.
2. Construir versiones derivadas en las que se introduzcan deliberadamente violaciones de capas, ciclos de dependencia y concentración excesiva de dependencias salientes.
3. Ejecutar el prototipo sobre la versión base y sobre cada versión alterada, manteniendo constante el mismo catálogo de reglas.

4. Comparar los resultados observados con la matriz de validación esperada para verificar si el mecanismo detecta cada deriva, si disminuye la **CR** cuando corresponde y si la clasificación del estado arquitectónico coincide con el escenario construido.

Los indicadores principales de evaluación serán los siguientes:

- **Cobertura de detección:** proporción de derivas inducidas que el prototipo logra identificar.
- **Falsos positivos en la línea base:** verificación de que el mecanismo no marque como anómalo un estado estructural esperado.
- **Variación de la CR:** comprobación de que la métrica sea sensible a la degradación introducida en las versiones alteradas.
- **Capacidad explicativa del reporte:** presencia de la regla violada, su ubicación y evidencia estructural suficiente para interpretación humana.

Con ello, el proyecto no solo mostrará que el prototipo funciona, sino también bajo qué criterios se evaluará su desempeño.

6.5. Resultados esperados

Como resultado del desarrollo del anteproyecto se espera obtener los siguientes productos verificables:

- Un prototipo funcional capaz de extraer dependencias estructurales desde el código fuente, construir un grafo y ejecutar reglas de conformidad arquitectónica.
- Un catálogo documentado de *Fitness Functions* para las tres familias de deriva seleccionadas, con su racionalidad, evidencia esperada y criterio de incumplimiento.
- Una matriz de validación y un conjunto de reportes que permitan observar la **CR**, las reglas incumplidas y la clasificación del estado arquitectónico de los escenarios evaluados.

- Evidencia experimental obtenida al comparar una línea base con versiones alteradas del caso de estudio, suficiente para juzgar la utilidad del mecanismo propuesto.
- Un documento final de investigación que sistematice el problema, la solución construida, el proceso metodológico y los hallazgos derivados de la evaluación.

La siguiente tabla establece la trazabilidad entre cada objetivo específico y el resultado esperado que lo concreta, de modo que sea verificable la cobertura y coherencia interna del proyecto.

Tabla 2: Trazabilidad entre objetivos específicos y resultados esperados

Objetivo específico	Resultado esperado asociado
OBJ1 — Formular el catálogo de <i>Fitness Functions</i>	Catálogo documentado con racionalidad, evidencia esperada y criterio de incumplimiento para cada regla de las tres familias de deriva seleccionadas.
OBJ2 — Implementar el componente de análisis estático	Componente de software que extrae dependencias estructurales desde el código fuente y genera el grafo de dependencia del sistema evaluado.
OBJ3 — Construir el motor de validación con CR	Motor de validación que calcula la Tasa de Conformidad (CR) y clasifica el estado arquitectónico (aceptable / en observación / no conforme), junto con la matriz de validación y los reportes de salida.
OBJ4 — Ejecutar el experimento controlado	Evidencia experimental: reportes comparativos entre la versión base y las versiones con derivas deliberadas, que demuestran la capacidad del mecanismo de detectar desviaciones y clasificar correctamente el estado arquitectónico.
Integrador (todos los objetivos)	Documento final de investigación que sistematiza el problema, la solución construida, el proceso metodológico y los hallazgos derivados de la evaluación experimental.

6.6. Criterios de cierre

Se considerará que la metodología ha producido resultados suficientes para el anteproyecto cuando se cumplan los siguientes criterios:

- Existencia de un catálogo acotado de reglas ejecutables.
- Disponibilidad de un prototipo funcional sobre un único ecosistema.
- Generación de reportes de conformidad a partir del código fuente.
- Evidencia de que el mecanismo identifica derivas introducidas en un entorno controlado.

Estos criterios permiten mantener una meta realista y medible, adecuada al tiempo de dedicación previsto para el trabajo.

6.7. Criterios de éxito

Los criterios de éxito se derivan directamente de los objetivos específicos del proyecto:

- **Éxito del objetivo 1:** existe un catálogo documentado con al menos una regla ejecutable por cada familia de deriva definida, y cada regla incluye propósito, evidencia esperada y criterio de incumplimiento.
- **Éxito del objetivo 2:** el componente de análisis estático construye un grafo navegable del sistema seleccionado y representa de forma consistente los componentes y dependencias necesarios para evaluar las reglas.
- **Éxito del objetivo 3:** el motor calcula automáticamente la **CR**, aplica los umbrales definidos y clasifica los escenarios evaluados de forma coherente con la matriz de validación del experimento.
- **Éxito del objetivo 4:** el experimento controlado permite detectar al menos el 80 % de las derivas introducidas deliberadamente, mantener la línea base sin hallazgos críticos inesperados y generar reportes comprensibles para apoyar decisiones de mantenimiento.

7. Cronograma

El proyecto se plantea para ejecutarse en **cinco meses**, con una dedicación promedio de **9 a 10 horas por semana**, para un total de **192 horas**. El cronograma adopta una lógica iterativa: cada mes cierra con una revisión de avance, ajuste del backlog técnico y validación de entregables con el director.

Mes	Periodo	Actividades principales	Horas	Entregable
Mes 1	Mayo 2026	Revisión bibliográfica focalizada, selección del ecosistema y del caso de estudio, definición del catálogo inicial de reglas, construcción de la matriz de validación y primera revisión con el director.	34	Protocolo de trabajo y catálogo inicial de reglas
Mes 2	Junio 2026	Diseño técnico del prototipo, definición de entradas y salidas, modelado del grafo de dependencia, preparación del entorno de desarrollo y construcción del extractor base.	38	Diseño del prototipo y extracción inicial de dependencias
Mes 3	Julio 2026	Implementación del componente de análisis estático, consolidación del grafo estructural, codificación de reglas de capas y de concentración de dependencias, pruebas iniciales sobre el caso base.	42	Prototipo funcional parcial con primeras reglas ejecutables
Mes 4	Agosto 2026	Implementación de detección de ciclos, cálculo de la Tasa de Conformidad (CR), generación de reportes, inyección controlada de derivas y ajustes del prototipo a partir de los resultados.	40	Prototipo funcional completo y reporte preliminar de conformidad
Mes 5	Septiembre 2026	Ejecución final del estudio de caso, análisis comparativo entre línea base y versiones alteradas, consolidación de resultados, redacción del informe de avances y preparación de la socialización.	38	Resultados analizados, conclusiones preliminares y documento consolidado
Total			192	

Tabla 3: Cronograma mensual propuesto para el anteproyecto

Para mantener el cronograma bajo control, al cierre de cada mes se realizará una

revisión de entre 1 y 2 horas con el director. Esa revisión permitirá contrastar horas planeadas frente a horas reales, depurar el alcance si aparece sobrecarga y priorizar el siguiente ciclo de trabajo sin comprometer los objetivos centrales del proyecto.

La distribución de horas por tipo de trabajo quedará así: **32 h** para revisión y definición de reglas, **80 h** para diseño e implementación del prototipo, **42 h** para evaluación controlada y ajustes, y **38 h** para documentación, reuniones y cierre. Esta distribución concentra el mayor esfuerzo en la construcción del artefacto, pero conserva tiempo suficiente para validar resultados y documentar el proceso de manera académicamente defendible.

Referencias

- Anthony, E., Berntsson, A., Santilli, T. & Wohlrab, R. (2024). We're Drifting Apart: Architectural Drift from the Developers' Perspective. *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA 2024)*, 101-111. <https://doi.org/10.1109/ICSA59870.2024.00018>
- Diaz-Pace, J. A., Soria, A., Rodriguez, G. & Campo, M. R. (2011). Taking ArchSync to the Real World: An Analysis of Three Case-Studies.
- Fitness Functions – Safeguard Architecture with Automated Checks. (s.f.). <https://continuous-architecture.org/practices/fitness-functions/>
- Hamed, A. S. E., Elbakry, H. M., Riad, A. E. & Moawad, R. (2023). A Proposed Technical Debt Management Approach Applied on Software Projects in Egypt. *Journal of Internet Services and Information Security (JISIS)*, 13(3), 156-177. <https://doi.org/10.58346/JISIS.2023.I3.010>
- Jolak, R., Karlsson, S. & Dobslaw, F. (2025). An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software*, 225, 112382. <https://doi.org/10.1016/j.jss.2025.112382>
- Klein, J., Nord, R. & Shull, F. (2022). *Why Architecture Conformance Matters for Software Systems* (inf. téc. N.º DM22-0924). Software Engineering Institute, Carnegie Mellon University.
- Make your architecture agile with fitness functions | Okoone. (s.f.). <https://www.okoone.com/spark/technology-innovation/make-your-architecture-agile-with-fitness-functions/>
- Manglaras, O., Farkas, A., Woolford, T., Treude, C. & Wagner, M. (2026). ModARO: A Modular Approach to Architecture Reconstruction of Distributed Microservice Codebases. <https://arxiv.org/pdf/2602.08181v1>
- Martín, D. S., Camargo, V. V. D. & Angulo, G. (s.f.). Architectural Conformance Checking for MAPE-K-based Self-Adaptive Systems.
- Modawski, A. & Wolniak, A. (2025). Lightweight AI Governance (LAIG) Framework for SMEs. *Proceedings of the European Workshop on Trustworthy AI*

- (*TRUST-AI 2025*), organized as part of the *European Conference of Artificial Intelligence (ECAI 2025)*. <https://ceur-ws.org/Vol-4132/short57.pdf>
- Murphy, G. C., Notkin, D. & Sullivan, K. J. (2001). Software Reflexion Models: Bridging the Gap between Source and High-Level Models. *IEEE Transactions on Software Engineering*, 27(4), 364-380. <https://doi.org/10.1109/32.908994>
- Paul, P. & Wang, R. (2019). Fitness function-driven development. *Thoughtworks*. <https://www.thoughtworks.com/insights/articles/fitness-function-driven-development>
- Pigazzini, I. (2021). *Evaluating and Detecting Architecture Erosion* (PhD Thesis). Università degli Studi di Milano-Bicocca.
- Sas, D. & Avgeriou, P. (2022). An architectural technical debt index based on machine learning and architectural smells [Bernoulli Institute, University of Groningen].
- Sas, D., Avgeriou, P. & Fontana, F. A. (2019). Investigating Instability Architectural Smells Evolution: An Exploratory Case Study. *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 557-567. <https://doi.org/10.1109/ICSME.2019.00090>
- Unit test your Java architecture - ArchUnit. (s.f.). <https://www.archunit.org/>